BANK ACCOUNT MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

NELAPUDI BHASKAR RAJU -25B11AI843
BHARGAV BHUPATIRAJU -25B11AI117
REDDIM SUBBARAO -25B11AIA27
GANDHAM SHYAMTA RAM -25B11AI312

Under the esteemed guidance of

Mrs. G L Narasamba Vanguri MTech, (Ph.D.)
Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))
Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY
(Formerly Aditya Engineering College(A))
Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled "Bank Account Management System" is being submitted by N BHASKAR RAJU (Roll No. 25B11AI843), BHARGAV BHUPATIRAJU (Roll No. 25B11AI117), R SUBBARAO (Roll No. 25B11AIA27) and G SHYAMTA RAM (Roll No. 25B11AI312) in
partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide
Mrs. V L Narasamba Vanguri, M. Tech, (Ph.D.)
Assistant professor

Head of the Department
Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled "Bank Account Management System" is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates
NELAPUDI BHASKAR RAJU -25B11AI843
BHARGAV BHUPATIRAJU -25B11AI117
REDDIM SUBBARAO -25B11AIA27
GANDHAM SHYAMTA RAM -25B11AI312

Date: 14 September 2026
Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates
NELAPUDI BHASKAR RAJU -25B11AI843
BHARGAV BHUPATIRAJU -25B11AI117
REDDIM SUBBARAO -25B11AIA27
GANDHAM SHYAMTA RAM -25B11AI312

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: Bank Account Management System

Submitted by:

S.No	Student Name	Roll Number
1	NELAPUDI BHASKAR RAJU	25B11AI843
2	BHARGAV BHUPATIRAJU	25B11AI117
3	REDDIM SUBBARAO	25B11AIA27
4	GANDHAM SHYAMTA RAM	25B11AI312

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27 Date of Submission: 14 September 2026

GitHub Repository Link: https://github.com/DBMS-2026/Bank-Account-Management-System-DBMS

1. ABSTRACT

The **Bank Account Management System** is a relational database project designed to manage core banking operations across multiple branches.

The system consists of six main entities: **Branch, Customer, Employee, Account, TransactionRecord, and Loan**, connected through foreign-key relationships to maintain data integrity.

Built using **SQL** and normalized up to **Third Normal Form (3NF)**, the system uses **NOT NULL, UNIQUE, and CHECK constraints** to maintain accurate and valid data.

The system supports customer management, savings/current/fixed-deposit accounts, deposits, withdrawals, transaction recording, and loan tracking. SQL queries demonstrate **joins, aggregate functions, GROUP BY/HAVING, subqueries, and reporting operations**.

TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	**ABSTRACT**	v
	List of Figures	vii
	List of Tables	vii
2	**INTRODUCTION**	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	**PROBLEM STATEMENT**	1
4	**OBJECTIVES**	1
5	**TOOLS & TECHNOLOGIES USED**	2
6	**DATABASE DESIGN**	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	**DATABASE IMPLEMENTATION**	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	**MODULES & FUNCTIONALITIES**	14
9	**GITHUB REPOSITORY & TEAM RESPONSIBILITIES**	14
10	**ENHANCEMENTS & FUTURE IMPROVEMENTS**	15
11	**CONCLUSION**	15
12	**REFERENCES**	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	4
Figure 1	Customer records / basic SELECT output	12
Figure 2	Active savings accounts / filtering output	12
Figure 3	Aggregate or GROUP BY output	13
Figure 4	Join or subquery output	13

LIST OF TABLES

Table No.	Title of the Table	Page No.
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	2
6.2	Entity Attributes	2
6.4	Relationships & Cardinality	4
6.5	Relational Database Schema	4
7.2	Sample Data — DML.	10
7.3	Constraints & Data Integrity	10
7.4	SQL Operations & Queries	10
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14

2. INTRODUCTION

2.1 Background

Banking systems handle large amounts of structured information related to customers, branches, accounts, employees, loans, and financial transactions. A relational database is required to store this information consistently and to maintain relationships between the different records. In this project, the banking domain is represented using six related tables: Branch, Customer, Employee, Account, Loan, and TransactionRecord. The schema separates data into appropriate entities and uses keys and constraints to reduce duplication and protect data integrity. SQL queries are then used to retrieve, update, aggregate, and analyse the stored information.

2.2 Purpose of the Database

The database provides a centralized structure for managing branch details, customer identity and contact information, employee details, account balances and statuses, loan information, and account transaction history. It can support banking staff and administrators who need to retrieve customer/account information, maintain balances, record transactions, monitor loans, and generate summary information. The relationships between the tables allow related information to be accessed through SQL joins and subqueries instead of maintaining disconnected records.

3. PROBLEM STATEMENT

Traditional branch-wise record keeping can lead to **duplicate customer information, inconsistent records and difficulty in tracking transactions**.

The proposed system centralizes branch, customer, employee, account, loan and transaction information into a **single relational database**.

The system provides structured data management, referential integrity, validation and SQL-based reporting for efficient banking operations.

4. OBJECTIVES

- Objective 1: Design a relational schema containing six related banking entities.
- Objective 2: Implement primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints for data integrity.
- Objective 3: Populate the database with representative branch, customer, employee, account, loan, and transaction records.
- Objective 4: Demonstrate SQL retrieval, updates, joins, aggregate functions, GROUP BY/HAVING, and subqueries.
- Objective 5: Document the database design, implementation, sample data, queries, and team responsibilities

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used
Database Server	SQ-compatible relational database
Query / Admin Tool	SQL client / SQL environment
Front-end (if any)	Not specified in the uploaded SQL files
Operating System	Not specified in the uploaded files
Version Control	GitHub

6. DATABASE DESIGN

6.1 Identified Entities

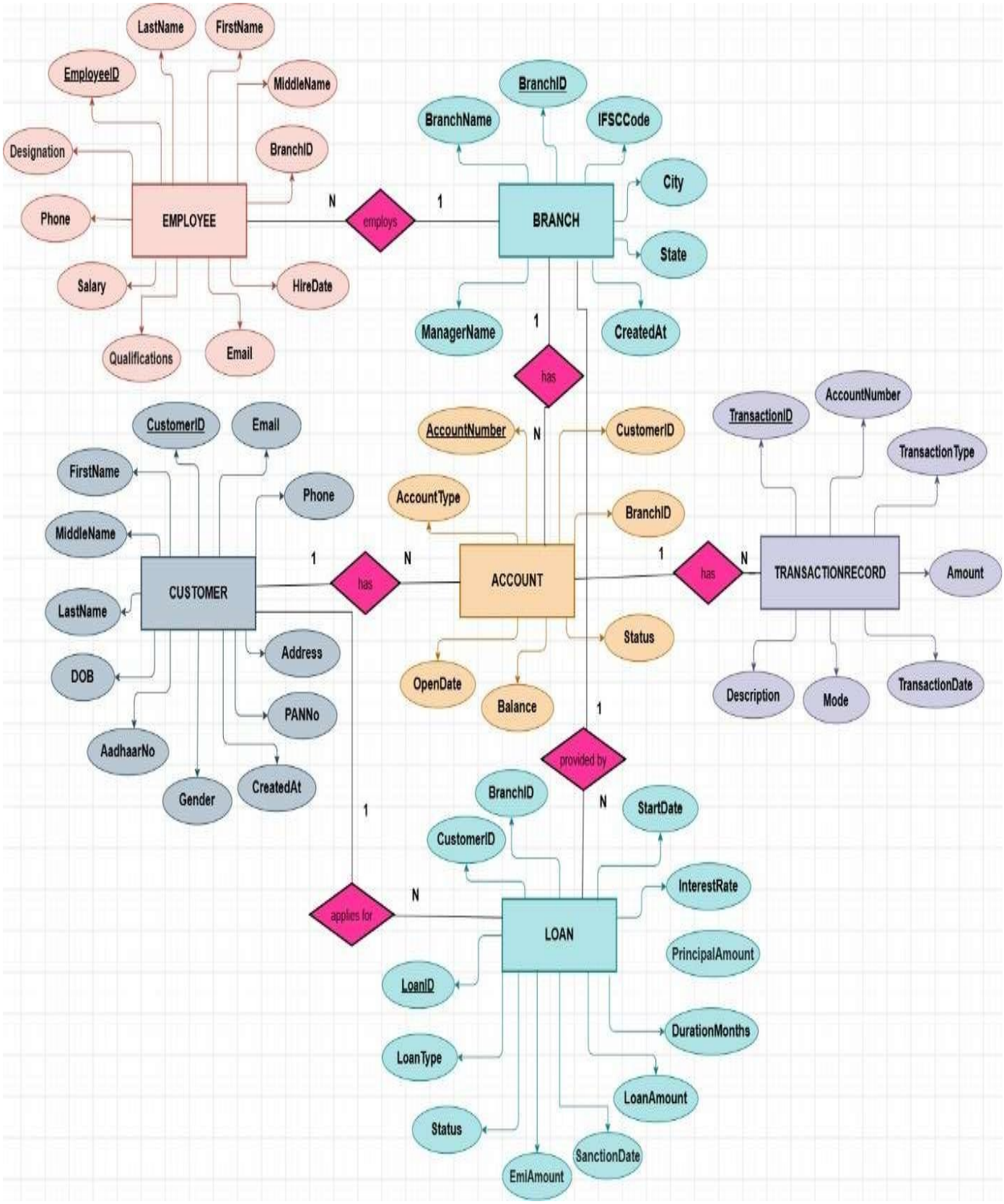
S.No	Entity Name	Description
1	Branch	Stores bank branch identification, location, IFSC code, manager, and creation date.
2	Customer	Stores customer identity, contact, KYC-related identifiers, address, and creation date.
3	Employee	Stores employee details, branch assignment, designation, qualification, salary, and contact information.
4	Account	Stores customer bank accounts, branch, account type, balance, opening date, and status.
5	TransactionRecord	Stores deposits, withdrawals, transfers, amounts, dates, modes, and descriptions for accounts.
6	Loan	Stores customer loans, branch, loan type, amounts, interest, duration, EMI, status, and sanction information.

6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Branch	BranchID	INTEGER	PK	Simple
Branch	BranchName	VARCHAR(50)	—	Simple
Branch	IFSCCode	VARCHAR(11)	—	Simple
Branch	City	VARCHAR(50)	—	Simple
Branch	State	VARCHAR(50)	—	Simple
Branch	ManagerName	VARCHAR(50)	—	Simple
Branch	CreatedAt	DATE	—	Derived/Default
Customer	CustomerID	INTEGER	PK	Simple
Customer	FirstName	VARCHAR(30)	—	Simple
Customer	MiddleName	VARCHAR(30)	—	Simple
Customer	LastName	VARCHAR(30)	—	Simple
Customer	DOB	DATE	—	Simple
Customer	Gender	VARCHAR(10)	—	Simple
Customer	AadhaarNo	VARCHAR(12)	—	Simple

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Customer	PANNo	VARCHAR(10)	—	Simple
Customer	Email	VARCHAR(100)	—	Simple
Customer	Phone	VARCHAR(12)	—	Simple
Customer	Address	VARCHAR(100)	—	Simple
Customer	CreatedAt	DATE	—	Derived/Default
Employee	EmployeeID	INTEGER	PK	Simple
Employee	BranchID	INTEGER	FK	Simple
Employee	FirstName	VARCHAR(30)	—	Simple
Employee	MiddleName	VARCHAR(30)	—	Simple
Employee	LastName	VARCHAR(30)	—	Simple
Employee	Designation	VARCHAR(50)	—	Simple
Employee	Qualifications	TEXT	—	Simple
Employee	Salary	DECIMAL(12,2)	—	Simple
Employee	Phone	VARCHAR(12)	—	Simple
Employee	Email	VARCHAR(100)	—	Simple
Employee	HireDate	DATE	—	Simple
Account	AccountNumber	VARCHAR(20)	PK	Simple
Account	CustomerID	INTEGER	FK	Simple
Account	BranchID	INTEGER	FK	Simple
Account	AccountType	VARCHAR(20)	—	Simple
Account	Balance	DECIMAL(15,2)	—	Simple
Account	OpenDate	DATE	—	Simple
Account	Status	VARCHAR(15)	—	Simple
TransactionRecord	TransactionID	INTEGER	PK	Simple
TransactionRecord	AccountNumber	VARCHAR(20)	FK	Simple
TransactionRecord	TransactionType	VARCHAR(15)	—	Simple
TransactionRecord	Amount	DECIMAL(15,2)	—	Simple
TransactionRecord	TransactionDate	DATE	—	Derived/Default
TransactionRecord	Mode	VARCHAR(20)	—	Simple
TransactionRecord	Description	VARCHAR(255)	—	Simple
Loan	LoanID	INTEGER	PK	Simple
Loan	CustomerID	INTEGER	FK	Simple
Loan	BranchID	INTEGER	FK	Simple
Loan	LoanType	VARCHAR(50)	—	Simple
Loan	LoanAmount	DECIMAL(15,2)	—	Simple
Loan	PrincipalAmount	DECIMAL(15,2)	—	Simple
Loan	InterestRate	DECIMAL(5,2)	—	Simple
Loan	StartDate	DATE	—	Simple
Loan	DurationMonths	INTEGER	—	Simple
Loan	EmiAmount	DECIMAL(10,2)	—	Simple
Loan	Status	VARCHAR(20)	—	Simple
Loan	SanctionDate	DATE	—	Simple

6.3 ER Diagram



6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Branch	employs	Employee	1:N
Branch	has	Account	1:N
Branch	handles	Loan	1:N
Customer	owns	Account	1:N
Customer	takes	Loan	1:N
Account	has	TransactionRecord	1:N

6.5 Relational Database Schema

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Branch	BranchID	INTEGER	PK	NOT NULL, FOREIGN KEY → Branch
Branch	BranchName	VARCHAR(50)	—	NOT NULL
Branch	IFSCCode	VARCHAR(11)	—	NOT NULL, UNIQUE, CHECK length=11
Branch	City	VARCHAR(50)	—	NOT NULL
Branch	State	VARCHAR(50)	—	NOT NULL
Branch	ManagerName	VARCHAR(50)	—	NOT NULL
Branch	CreatedAt	DATE	—	DEFAULT CURRENT_DATE
Customer	CustomerID	INTEGER	PK	NOT NULL, FOREIGN KEY → Customer
Customer	FirstName	VARCHAR(30)	—	NOT NULL
Customer	MiddleName	VARCHAR(30)	—	NULL allowed
Customer	LastName	VARCHAR(30)	—	NOT NULL
Customer	DOB	DATE	—	NOT NULL
Customer	Gender	VARCHAR(10)	—	CHECK Male/Female/Other
Customer	AadhaarNo	VARCHAR(12)	—	NOT NULL, UNIQUE, CHECK length=12
Customer	PANNo	VARCHAR(10)	—	NOT NULL, UNIQUE, CHECK length=10
Customer	Email	VARCHAR(100)	—	NOT NULL, UNIQUE
Customer	Phone	VARCHAR(12)	—	NOT NULL
Customer	Address	VARCHAR(100)	—	NOT NULL
Customer	CreatedAt	DATE	—	DEFAULT CURRENT_DATE
Employee	EmployeeID	INTEGER	PK	PRIMARY KEY
Employee	BranchID	INTEGER	FK	NOT NULL, FOREIGN KEY → Branch
Employee	FirstName	VARCHAR(30)	—	NOT NULL
Employee	MiddleName	VARCHAR(30)	—	NULL allowed
Employee	LastName	VARCHAR(30)	—	NOT NULL
Employee	Designation	VARCHAR(50)	—	NOT NULL
Employee	Qualifications	TEXT	—	NULL allowed

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Employee	Salary	DECIMAL(12,2)	—	NOT NULL, CHECK >= 0
Employee	Phone	VARCHAR(12)	—	NOT NULL
Employee	Email	VARCHAR(100)	—	NOT NULL, UNIQUE
Employee	HireDate	DATE	—	NOT NULL
Account	AccountNumber	VARCHAR(20)	PK	NOT NULL, FOREIGN KEY → Account
Account	CustomerID	INTEGER	FK	NOT NULL, FOREIGN KEY → Customer
Account	BranchID	INTEGER	FK	NOT NULL, FOREIGN KEY → Branch
Account	AccountType	VARCHAR(20)	—	NOT NULL, CHECK Savings/Current/Fixed Deposit
Account	Balance	DECIMAL(15,2)	—	NOT NULL, DEFAULT 0, CHECK >= 0
Account	OpenDate	DATE	—	NOT NULL
Account	Status	VARCHAR(15)	—	NOT NULL, DEFAULT Active, CHECK allowed statuses
TransactionRecord	TransactionID	INTEGER	PK	PRIMARY KEY
TransactionRecord	AccountNumber	VARCHAR(20)	FK	NOT NULL, FOREIGN KEY → Account
TransactionRecord	TransactionType	VARCHAR(15)	—	NOT NULL, CHECK Deposit/Withdrawal/Transfer
TransactionRecord	Amount	DECIMAL(15,2)	—	NOT NULL, CHECK > 0
TransactionRecord	TransactionDate	DATE	—	DEFAULT CURRENT_DATE
TransactionRecord	Mode	VARCHAR(20)	—	NOT NULL, CHECK Online/ATM/Branch/UPI/Net Banking
TransactionRecord	Description	VARCHAR(255)	—	NULL allowed
Loan	LoanID	INTEGER	PK	PRIMARY KEY
Loan	CustomerID	INTEGER	FK	NOT NULL, FOREIGN KEY → Customer
Loan	BranchID	INTEGER	FK	NOT NULL, FOREIGN KEY → Branch
Loan	LoanType	VARCHAR(50)	—	NOT NULL, CHECK allowed loan types
Loan	LoanAmount	DECIMAL(15,2)	—	NOT NULL, CHECK > 0
Loan	PrincipalAmount	DECIMAL(15,2)	—	CHECK > 0
Loan	InterestRate	DECIMAL(5,2)	—	NOT NULL, CHECK 0–100
Loan	StartDate	DATE	—	NOT NULL
Loan	DurationMonths	INTEGER	—	NOT NULL, CHECK > 0
Loan	EmiAmount	DECIMAL(10,2)	—	CHECK >= 0
Loan	Status	VARCHAR(20)	—	NOT NULL, DEFAULT Active, CHECK allowed statuses
Loan	SanctionDate	DATE	—	NULL allowed

6.6 Normalization

UNF → 1NF → 2NF → 3NF

UNF: Banking information could be represented as a large combined record containing branch, customer, account, loan, employee, and transaction details.

1NF: The design separates repeating groups and keeps attributes atomic. For example, customer names are split into FirstName, MiddleName, and LastName, and Address is stored as one field.

2NF: Each entity is separated into its own table with a primary key so non-key attributes depend on the complete key.

3NF: Relationships are represented using foreign keys, so branch, customer, account, loan, employee, and transaction facts are stored in their appropriate tables rather than being repeatedly duplicated. This reduces update, insertion, and deletion anomalies.

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

```
    -- Drop tables if they exist
DROP TABLE IF EXISTS TransactionRecord;
DROP TABLE IF EXISTS Loan;
DROP TABLE IF EXISTS Account;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Customer;
DROP TABLE IF EXISTS Branch;

-- ------------------------------------------------------------------------------------------------------------------------
-- 1. BRANCH TABLE
-- ------------------------------------------------------------------------------------------------------------------------
CREATE TABLE Branch (
    BranchID INTEGER PRIMARY KEY,
    BranchName VARCHAR(50) NOT NULL,
    IFSCCode VARCHAR(11) NOT NULL UNIQUE,
    City VARCHAR(50) NOT NULL,
    State VARCHAR(50) NOT NULL,
    ManagerName VARCHAR(50) NOT NULL,
    CreatedAt DATE DEFAULT CURRENT_DATE,
    CONSTRAINT chk_ifsc_len CHECK (LENGTH(IFSCCode) = 11)
);

-- ------------------------------------------------------------------------------------------------------------------------
-- 2. CUSTOMER TABLE (Atomic attributes, 1NF/2NF/3NF compliant)
-- ------------------------------------------------------------------------------------------------------------------------
CREATE TABLE Customer (
    CustomerID INTEGER PRIMARY KEY,
    FirstName VARCHAR(30) NOT NULL,
    MiddleName VARCHAR(30),
    LastName VARCHAR(30) NOT NULL,
    DOB DATE NOT NULL,
    Gender VARCHAR(10) CHECK (Gender IN ('Male', 'Female', 'Other')),
    AadhaarNo VARCHAR(12) NOT NULL UNIQUE,
    PANNo VARCHAR(10) NOT NULL UNIQUE,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(12) NOT NULL,
    Address VARCHAR(100) NOT NULL,
    CreatedAt DATE DEFAULT CURRENT_DATE,
    CONSTRAINT chk_aadhaar CHECK (LENGTH(AadhaarNo) = 12),
    CONSTRAINT chk_pan CHECK (LENGTH(PANNo) = 10)
);

-- ------------------------------------------------------------------------------------------------------------------------
-- 3. EMPLOYEE TABLE
-- ------------------------------------------------------------------------------------------------------------------------
CREATE TABLE Employee (
    EmployeeID INTEGER PRIMARY KEY,
    BranchID INTEGER NOT NULL,
    FirstName VARCHAR(30) NOT NULL,
    MiddleName VARCHAR(30),
    LastName VARCHAR(30) NOT NULL,
    Designation VARCHAR(50) NOT NULL,
    Qualifications TEXT,
    Salary DECIMAL(12, 2) NOT NULL CHECK (Salary >= 0),
    Phone VARCHAR(12) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    HireDate DATE NOT NULL,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);
```

```
--- ----------------------------------------------------------------------------------------------------------------------------------------------------
-- 4. ACCOUNT TABLE
--- ----------------------------------------------------------------------------------------------------------------------------------------------------
CREATE TABLE Account (
    AccountNumber VARCHAR(20) PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    BranchID INTEGER NOT NULL,
    AccountType VARCHAR(20) NOT NULL CHECK (AccountType IN ('Savings', 'Current',
'Fixed Deposit')),
    Balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (Balance >= 0),
    OpenDate DATE NOT NULL,
    Status VARCHAR(15) NOT NULL DEFAULT 'Active' CHECK (Status IN ('Active',
'Inactive', 'Frozen', 'Closed')),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);

--- ----------------------------------------------------------------------------------------------------------------------------------------------------
-- 5. TRANSACTION TABLE
--- ----------------------------------------------------------------------------------------------------------------------------------------------------
CREATE TABLE TransactionRecord (
    TransactionID INTEGER PRIMARY KEY,
    AccountNumber VARCHAR(20) NOT NULL,
    TransactionType VARCHAR(15) NOT NULL CHECK (TransactionType IN ('Deposit',
'Withdrawal', 'Transfer')),
    Amount DECIMAL(15, 2) NOT NULL CHECK (Amount > 0),
    TransactionDate DATE DEFAULT CURRENT_DATE,
    Mode VARCHAR(20) NOT NULL CHECK (Mode IN ('Online', 'ATM', 'Branch', 'UPI', 'Net
Banking')),
    Description VARCHAR(255),
    FOREIGN KEY (AccountNumber) REFERENCES Account(AccountNumber)
);

--- ----------------------------------------------------------------------------------------------------------------------------------------------------
-- 6. LOAN TABLE
--- ----------------------------------------------------------------------------------------------------------------------------------------------------
CREATE TABLE Loan (
    LoanID INTEGER PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    BranchID INTEGER NOT NULL,
    LoanType VARCHAR(50) NOT NULL CHECK (LoanType IN ('Home', 'Car', 'Personal',
'Education', 'Business')),
    LoanAmount DECIMAL(15, 2) NOT NULL CHECK (LoanAmount > 0),
    PrincipalAmount DECIMAL(15, 2) CHECK (PrincipalAmount > 0),
    InterestRate DECIMAL(5, 2) NOT NULL CHECK (InterestRate >= 0 AND InterestRate <=
100),
    StartDate DATE NOT NULL,
    DurationMonths INTEGER NOT NULL CHECK (DurationMonths > 0),
    EmiAmount DECIMAL(10, 2) CHECK (EmiAmount >= 0),
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (Status IN ( 'Approved',
'Pending', 'Closed', 'Rejected', 'Active', 'Default')),
    SanctionDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);
```

7.2 Sample Data — DML.

Table	No. of Records	Sample Data / Screenshot Reference
Branch	5	dml.sql — Branch seed records
Customer	10	dml.sql — Customer seed records
Employee	12	dml.sql — Employee seed records

Account	15	dml.sql — Account seed records
Loan	5	dml.sql — Loan seed records
TransactionRecord	50	dml.sql — Transaction seed records

7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All six tables	Uniquely identifies each record and prevents duplicate key values.	ddl.sql
FOREIGN KEY	Employee, Account, Loan, TransactionRecord	Maintains valid relationships between related tables.	ddl.sql
NOT NULL	Required fields across tables	Prevents missing values in required attributes.	ddl.sql
UNIQUE	Branch.IFSCCode; Customer.AadhaarNo/PANNo/Email; Employee.Email	Prevents duplicate unique identifiers/contact values.	ddl.sql
CHECK	Gender, AccountType, Balance, Status, transaction type/mode, loan values, lengths	Restricts invalid or out-of-range values.	ddl.sql
DEFAULT	Branch.CreatedAt; Customer.CreatedAt; Account.Balance/Status; TransactionDate; Loan.Status	Provides default values when not explicitly supplied.	ddl.sql

7.4 SQL Operations & Queries

.

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	Basic SELECT — all customers	Retrieve customer records.	queries(2).sql Q1
2	Filtered SELECT — active savings accounts	Find active savings accounts ordered by balance.	queries(2).sql Q2
3	LIKE filtering	Search customers by name or email domain.	queries(2).sql Q3
4	Balance lookup	Check the balance of account 10001.	queries(2).sql Q4
5	UPDATE + INSERT	Deposit ₹5,000 into account 10001 and record the transaction.	queries(2).sql Q5
6	UPDATE + INSERT	Withdraw ₹1,000 when sufficient balance exists and record the transaction.	queries(2).sql Q6
7	Transaction history	Display account transaction history in descending date order.	queries(2).sql Q7
8	SUM aggregate	Calculate total funds across all accounts.	queries(2).sql Q8

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
9	GROUP BY + aggregates	Calculate account count, average balance, and total balance by account type.	queries(2).sql Q9
10	GROUP BY + aggregates	Calculate active loan count, total sanctioned amount, and average interest rate by loan type.	queries(2).sql Q10
11	JOIN + GROUP BY + HAVING	Find branches whose total account balance exceeds ₹500,000.	queries(2).sql Q11
12	JOIN + GROUP BY + HAVING	Find customers holding more than one account.	queries(2).sql Q12
13	Subquery	Find customers whose total account balance exceeds the average account balance.	queries(2).sql Q13
14	Subquery + ORDER BY + LIMIT	List transactions for the highest-balance account.	queries(2).sql Q14

7.5 Sample Outputs / Screenshots

Figure 1: Customer records / basic SELECT output

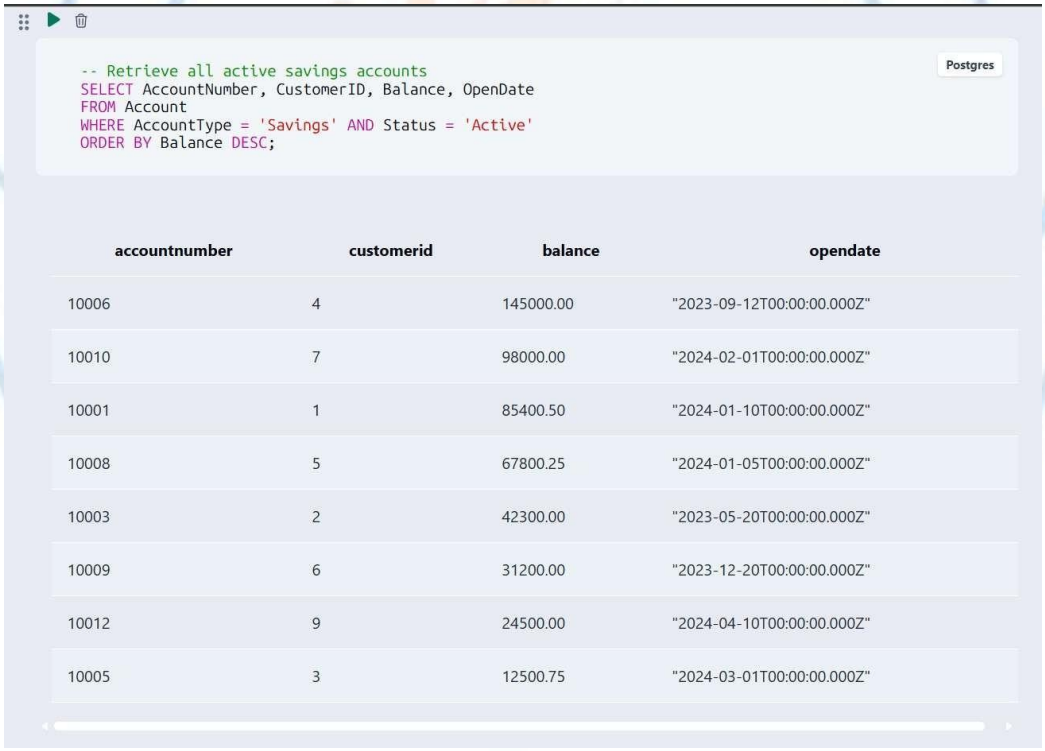


Figure 2: Active savings accounts / filtering output

```
-- Find customers whose total account balance is higher than the average balance across all bank
SELECT CustomerID, FirstName, LastName, Email
FROM Customer
WHERE CustomerID IN (
    SELECT CustomerID
    FROM Account
    GROUP BY CustomerID
    HAVING SUM(Balance) > (SELECT AVG(Balance) FROM Account)
);
```

Postgres

customerid	firstname	lastname	email
1	Gopi	Peddinti	gopi.p@aditya.ac.in
2	Aditya	Verma	aditya.verma@example.com
4	Bhavana	Kothapalli	bhavana.k@example.com
10	Manohar	Rawat	manohar.rawat@example.com

Figure 3: Aggregate or GROUP BY output

```
--Average account balance grouped by Account Type
SELECT AccountType, COUNT(*) AS AccountCount, AVG(Balance) AS AverageBalance, SUM(Balance) AS TotalTypeBa
FROM Account
GROUP BY AccountType;
```

Postgres

accounttype	accountcount	averagebalance	totaltypebalance
Fixed Deposit	3	550000.000000000000	1650000.00
Savings	8	63337.687500000000	506701.50
Current	4	207500.000000000000	830000.00

Figure 4: Join or subquery output

```
-- Find branches where total account balance exceeds Rs. 500,000
SELECT
    b.BranchName,
    SUM(a.Balance) AS TotalBalance
FROM Branch b
JOIN Account a
    ON b.BranchID = a.BranchID
GROUP BY b.BranchID, b.BranchName
HAVING SUM(a.Balance) > 500000;
```

Postgres

branchname	totalbalance
Hyderabad Corp	1563000.00
Vizag Branch	573500.00

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality
1	Branch Management	Stores and organizes branch identification, IFSC, location, manager, and creation information.
2	Customer Management	Maintains customer identity, KYC-related identifiers, contact details, and address.
3	Employee Management	Maintains employees and their association with branches, including designation, qualification, salary, and contact information.
4	Account Management	Stores account ownership, branch, account type, balance, opening date, and status.
5	Transaction Management	Records deposits, withdrawals, and transfers with amount, date, mode, and description.
6	Loan Management & Analysis	Stores loan details and supports loan and banking analysis through aggregate and grouped SQL queries.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: https://github.com/DBMS-2026/Bank-Account-Management-System-DBMS

Student Name	Roll Number	Role / Responsibility	Contribution Status
P GOPI	25B11AI924	Wrote DDL scripts (tables, keys, constraints)	Completed
G JASWANTH	25B11AI333	Designed ER diagram & relational schema	Completed
CH ESWAR	25B11AI186	Wrote CRUD, Join & Aggregate queries	Completed
CH CHARAN	25B11AI223	Compiled documentation & screenshots	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Future improvements can extend the current database with stronger transaction controls, user authentication and role-based access, richer reporting and dashboards, automated loan EMI calculations, audit logging, stored procedures/triggers where supported, indexing for frequently queried columns, and a front-end interface for banking staff. The uploaded SQL files do not include these additional features, so they are proposed enhancements rather than implemented features in the current submission.

11. CONCLUSION

The Bank Account Management System successfully models the main banking entities and their relationships using a relational database structure. The DDL defines six tables with primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. The DML script supplies representative records for branches, customers, employees, accounts, loans, and transactions. The query script demonstrates selection and filtering, balance updates, transaction logging, joins, aggregate functions, GROUP BY/HAVING, and subqueries. The project therefore meets its core objectives of relational design, data integrity, sample-data population, and practical SQL analysis. The team also divided the work across DDL development, ER/schema design, query development, and documentation/screenshot compilation.

12. REFERENCES

1. Uploaded project DDL script: ddl.sql.
2. Uploaded project DML/sample-data script: dml.sql.
3. Uploaded SQL query script: queries.sql.
4. Aditya University — DBMS Capstone Project documentation template.
5. GitHub repository: Bank Account Management System (project repository).

APPENDIX

A. Complete SQL Script

The final SQL implementation is supplied through the uploaded project files:

• ddl.sql — database table definitions, keys, foreign keys, and constraints.

• dml.sql — sample data for all six tables.

• queries.sql — representative SQL operations and analytical queries.

Repository: https://github.com/gopipeddinti1986-oss/Bank-Account-Management-System.git

B. Additional Screenshots / Outputs

Additional evidence can include SQL execution screenshots showing table creation, sample data, customer/account retrieval, balance updates, aggregate results, joins, GROUP BY/HAVING results, and subquery outputs. No execution screenshots were included in the uploaded source files, so this section is intentionally left for the team to add its actual screenshots.